
Continuous Trajectory Optimization with Machine Learning Techniques

Lirui Wang

University of Washington

Seattle, USA

`liruiw@cs.washington.edu`

Abstract

1 Trajectory optimization-based motion planning in robotics aims to find a feasible,
2 optimal trajectory between two fixed end points. Feasibility evaluates task-specific
3 constraints such as collision and joint limits, and optimality often requires the
4 trajectory to be smooth. A drawback of traditional optimization methods is that
5 they require a fine discretization of the trajectory to carefully reason about cost
6 information. In this work, we investigate how machine learning techniques can be
7 applied in trajectory optimization-based motion planners. Specifically, we will dis-
8 cuss the ideas of [5] and [7] that enable efficient continuous trajectory optimization.
9 While [5] introduce functional gradient descent trajectory optimization algorithm
10 for robot motion planning in Reproducing Kernel Hilbert Spaces (RKHSs), [7] for-
11 mulate motion planning as probabilistic inference problem in Gaussian Processes
12 (GPs).

1 Introduction and Related Works

14 Trajectory optimization starts with a possibly infeasible trajectory and then optimizes the trajectory
15 by minimizing a cost function. CHOMP [8] and related methods [2] optimize a cost functional
16 using covariant gradient descent, while STOMP [4] uses stochastic sampling of noisy trajectories to
17 optimize non-differentiable costs. More recently, TrajOpt [9] solves a sequential quadratic program
18 and performs convex continuous-time collision checking. While these optimization-based methods
19 have shown improvement on planning time and performance, in practice a fine discretization of the
20 trajectory is necessary to integrate cost information when reasoning about small obstacles.

21 A continuous-time trajectory representation can avoid some of these challenges and accelerate the
22 planning. Machine learning ideas provide a compact representation to handle continuous-time
23 trajectory. In this work, we summarize the contributions of [5] and [7] and provide some insights and
24 discussions of their ideas. RKHSs [5] provides natural smoothness in function approximation without
25 a large number of waypoints in parametrization. Smooth trajectories are represented as vectors in an
26 RKHS and different kernels can lead to different notions of smoothness. They propose to minimize
27 the cost functional regularized by the RKHS norm and demonstrate that it can be optimized efficiently
28 through taking larger steps than waypoint parametrization.

29 On the other hand, GPMP2 [7] considers the duality between optimization and inference, and models
30 planning as a inference problem, which captures optimality by a prior on trajectories and encodes
31 feasibility in a likelihood function. The Bayesian inference idea allows them to perform efficient
32 inference on factor graphs by solving least square problems. Using GP interpolation, the trajectory
33 can be queried at any point of interest. Moreover, they conduct simulation and real experiments to
34 show that the method not only demonstrates high success rate but also improves the planning time
35 dramatically.

2 Preliminaries

2.1 Trajectory Optimization

A trajectory is a function $\xi : [0, 1] \mapsto \mathcal{C}$ that maps time $t \in [0, 1]$ to robot configuration space $\mathcal{C} \subset \mathbb{R}^d$. $\xi(t)$ defines the robot configuration at time t . The goal of motion planning is to find trajectory that satisfies constraints and minimizes costs, so we can formulate the objective as

$$\min \mathcal{F}(\xi(t))$$

where the functional $\mathcal{F}$ often contains measure of optimality such as smoothness in terms of velocity, acceleration, and jerks etc. and measure of feasibility like proximity to nearest obstacles. For the purpose of this review, we ignore trajectory constraints such as joint limits.

2.2 Reproducing Kernel Hilbert Space

For a compact subset $\mathcal{X} \subset \mathbb{R}^d$ and a Hilbert space $\mathcal{H}$ of functions f , we say $\mathcal{H}$ is a Reproducing Kernel Hilbert Space (RKHS) if there exists K such that K 's span is dense in $\mathcal{H}$ and K is a reproducing kernel. We say $K(x, y)$ is a reproducing kernel of a Hilbert space $\mathcal{H}$ if for any $f \in \mathcal{H}$, we have

$$f(x) = \langle K(x, \cdot), f(\cdot) \rangle_{\mathcal{H}} = \langle f(\cdot), K(\cdot, x) \rangle_{\mathcal{H}} \quad (1)$$

An example of K could be Dirac delta function $\delta_y(x)$. Let g and $h \in \mathcal{H}$ be:

$$g(\cdot) = \sum_{i=1}^M \alpha_i K(\cdot, x_i), f(\cdot) = \sum_{j=1}^N \beta_j K(\cdot, x_j) \quad (2)$$

We define the inner product to be

$$\langle g, h \rangle_{\mathcal{H}} = \sum_{i,j} \alpha_i \beta_j K(x_i, x_j) \quad (3)$$

2.3 Gaussian Process

A random process $(X_t)_{t \in T}$ is called a Gaussian Process (GP) if for any finite subset $T_0 \subset T$, the random vector $(X_t)_{t \in T_0}$ has a normal distribution. Using the GP framework, we can say that for any collection of times $t = \{t_0, \dots, t_N\}$, ξ has a joint Gaussian distribution:

$$\xi = [\xi_0, \dots, \xi_N] \sim \mathcal{N}(\mu, K) \quad (4)$$

A Gaussian process is fully describe by its mean μ and covariance kernel K , defined as

$$\mu = [\mu_0, \dots, \mu_N], K = [K(t_i, t_j)]_{i,j, 0 \leq i, j \leq N} \quad (5)$$

A specific structured kernel can be generated by linear time-varying stochastic differential equation (LTV-SDE)

$$\dot{\xi}(t) = A(t)\xi(t) + u(t) + F(t)w(t) \quad (6)$$

where $u(t)$ is the known system control input, $A(t)$ and $F(t)$ are time varying matrices of the system, and $w(t)$ is itself a zero-mean GP: $w(t) \sim \mathcal{GP}(0, Q_C \delta(t - t'))$. Q_C denotes the power-spectral density matrix.

2.4 Factor Graph

[1] A factor graph $G = \{\mathcal{U}, \mathcal{V}, \mathcal{E}\}$ is a bipartite graph with factor nodes $\phi_i \in \mathcal{U}$ and variable nodes $x_j \in \mathcal{V}$. Edges $e_{ij} \in \mathcal{E}$ are always between factor nodes and variable nodes. We write the set of variable nodes adjacent to a factor ϕ_i as $\mathcal{N}(\phi_i)$ and X_i for an assignment to this set. A factor graph F allows us to specify a joint density as a product of factors by defining the factorization of a global function $\phi(X)$ as

$$\phi(X) = \prod_i \phi_i(X_i) \quad (7)$$

Note that the independence relationships are encoded by the edges e_{ij} of the factor graph, with each factor ϕ_i a function of only the variables X_i in its adjacency set $\mathcal{N}(\phi_i)$.

68 3 RKHSs

69 A set of trajectories can be treated as a Hilbert space by defining vector-space operations of trajectories
 70 such as addition and scalar multiplication. Moreover, it can be upgraded to a vector-valued RKHS
 71 $\mathcal{H}$ by assuming the functional $\xi \mapsto \langle \xi(t), q \rangle$ is continuous for arbitrary $q \in \mathcal{C}$ and $t \in [0, 1]$. The
 72 norm of a trajectory in the RKHS measures the size, and the correlation among trajectories quantifies
 73 the complexity of the trajectory. The RKHS representation of trajectories allows efficient trajectory
 74 optimization through functional gradient descent.

75 3.1 RKHS Formulation

76 Assume our configuration space $\mathcal{C}$ is d dimension, the trajectories form an RKHS of vector-valued
 77 functions, and the associated reproducing kernel becomes a matrix valued kernel $K : [0, 1] \times [0, 1] \mapsto$
 78 $\mathcal{C} \times \mathcal{C}$.

$$K(t, t') = \begin{bmatrix} k_{1,1}(t, t') & k_{1,2}(t, t') & \dots & k_{1,d}(t, t') \\ k_{2,1}(t, t') & k_{2,2}(t, t') & \dots & k_{2,d}(t, t') \\ \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots \\ k_{d,1}(t, t') & k_{d,2}(t, t') & \dots & k_{d,d}(t, t') \end{bmatrix} \quad (8)$$

79 Eq. 8 defines the kernel matrix of joint interaction between timesteps t and t' . It has a intuitive
 80 physical interpretation as an inertial matrix of a rigid body changing over time. $k_{i,j}(t, t')$ represents
 81 the correlation between two configurations, i.e. how $\xi_i(t)$ affect $\xi_j(t')$. Overall, the matrix represents
 82 that perturbations of given joints propagate through time as well as rest of the joints. Since we have a
 83 vector-valued RKHS, for all $y \in \mathcal{C}$, equation 1 becomes

$$y^T \xi(t) = \langle \xi, K(t, \cdot) y \rangle_{\mathcal{H}} \quad (9)$$

84 Since a trajectory in RKHS admits a representation in terms of finite support $\{t_i\}_{i=1}^N \in \mathcal{T}$, we can
 85 write ξ as

$$y^T \xi(\cdot) = \sum_{i=1}^N a_i K(\cdot, t_i) y \quad (10)$$

86 where a_i is a coefficient vector. Consider a special case where we choose the $y = e_k$ to be the
 87 indicator vector of joint k . Then we can represent the inner product in $\mathcal{H}$ of two trajectory functions
 88 $\xi^1(\cdot) = \sum_{i,k} a_{i,k} k(t_i, \cdot) e_k$ and $\xi^2(\cdot) = \sum_{j,k} b_{j,k} k(t_j, \cdot) e_k$ as

$$\langle \xi^1, \xi^2 \rangle_{\mathcal{H}} = \sum_k \langle \xi_k^1, \xi_k^2 \rangle_{\mathcal{H}} = \sum_k \sum_{i,j} a_{i,k} b_{j,k} k(t_i, t_j) \quad (11)$$

89

$$\|\xi\|_{\mathcal{H}} = \sum_k \langle \xi, \xi \rangle_{\mathcal{H}} = \sum_k \sum_{i,j} a_{i,k} a_{j,k} k(t_i, t_j) \quad (12)$$

90 One kernel example is Gaussian kernel $k(t, t') = \exp(\|t - t'\|^2) / 2\sigma^2$ for some smoothing bandwidth
 91 σ . Then a trajectory is a weighted sum of radial basiss functions.

$$\xi(t) = \sum_{i,k} a_{i,k} \exp(\|t - t_i\|^2) / 2\sigma^2 e_k \quad (13)$$

92 where the coefficients $a_{i,k}$ assess how important a particular joint k at time t_i is to the overall trajectory.
 93 As a linear smoother, RKHS represents a trajectory as a weighted sum of local perturbations to the
 94 motions of different joints centered at different times.

95 3.2 Functional Gradient Descent

96 We define a cost function $\mathcal{H} \mapsto \mathbb{R}$ for trajectory optimization, which balances between a regularization
 97 term in size of the trajectory and the proximity to obstacles:

$$\mathcal{U}[\xi] = \mathcal{U}_{obs}[\xi] + \frac{\beta}{2} \|\xi\|_{\mathcal{H}}^2 \quad (14)$$

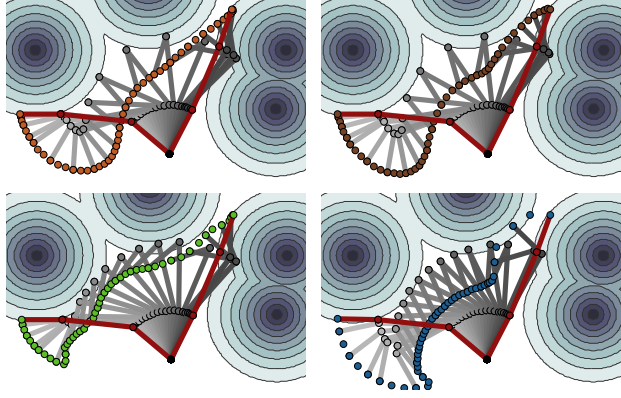


Figure 1: In the experiments [5], they use 100 different random obstacle placements and fix start and goal configuration as experiment setups. This is a 3DoF Robot in C-space. Trajectory after 10 iterations: top-left: Gaussian RBF kernel, top-right: B-splines kernel, bottom-left: Laplaceian RBF kernel, bottom-right: Waypoints

98 We can then derive the functional gradient update at step n within an ellipsoid trust region about ξ^n
 99 in the RKHS:

$$\xi^{n+1} = \underset{\xi}{\operatorname{argmin}} \langle \xi - \xi^n, \nabla \mathcal{U}[\xi^n] \rangle_{\mathcal{H}} + \frac{\lambda}{2} \|\xi - \xi^n\|_{\mathcal{H}}^2 \quad (15)$$

100 The closed form solution to the minimization above can then be computed. Define the set of time
 101 points $\mathcal{T}$ that support the trajectory contribute to the design of the kernel metric.

$$\|\xi\|_{\mathcal{H}}^2 = a^T K(\mathcal{T}, \mathcal{T}) a \quad (16)$$

102 Here a is the concatenation of all coefficients a_i over $\mathcal{T}$. $K(\mathcal{T}, \mathcal{T})$ can be interpreted as a tensor
 103 matrix in Riemannian manifold and its inverse is the key element that bridges the functional gradient
 104 in RKHS and the functional gradient in Euclidean space

$$\nabla \mathcal{U} = K^{-1}(\mathcal{T}, \mathcal{T}) \nabla_E \mathcal{U} \quad (17)$$

105 They conduct an ablation experiment on kernel function, shown in Fig. 1. It's observed that Waypoints
 106 and Laplacian RBF kernels with large widths have similar behavior, while Gaussian RBF and B-
 107 splines kernels provide a smooth parameterization that allows the algorithm to take larger steps at
 108 each iteration.

109 4 GPs

110 Through GP, motion planning can be viewed as probabilistic inference. we seek to find a trajectory
 111 parameterized by ξ given desired events e . In this work, the desired event is collision free trajectory.
 112 The posterior density of ξ given e can be computed by Bayes rule from a prior and likelihood

$$p(\xi|e) \propto p(\xi)p(e|\xi) \quad (18)$$

113 where $p(\xi)$ is the prior on ξ that encourages smooth trajectories, and $p(e|\xi)$ is the likelihood that the
 114 trajectory ξ is collision free. The optimal trajectory ξ is then found by the maximum a posteriori
 115 (MAP) estimator. With a further formulation of inference as least squares optimization, they can
 116 perform inference very efficiently, so motion planning is extremely fast.

117 4.1 Gaussian Interpolation

118 [6] The solution to the initial problem of the LTV-SDE in Eq. 6 is

$$\xi(t) = \Phi(t, t_0)\xi_0 + \int_{t_0}^t \Phi(t, s)(u(s) + F(s)w(s))ds \quad (19)$$

119 where $\Phi(t, s)$ is the state transition matrix that transfers state from time s to time t . The mean μ and
 120 covariance function K of the GP defined by this LTV-SDE conditioned on an observation goal state

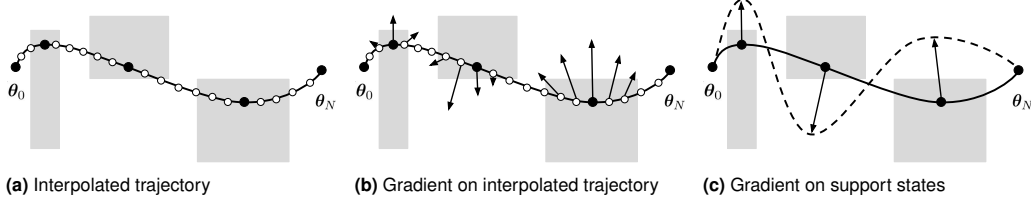


Figure 2: [6] An example showing how GP interpolation is used during optimization. (a) shows the current iteration of the trajectory (black curve) parameterized by a sparse set of support states (black circles) and interpolated states (white circles) by GP regression. Then, in (b) cost is evaluated on all states and their gradients are illustrated by the arrows. Finally, in (c) the cost and gradient information is propagated to just the support states illustrated by the larger arrows such that only the support states are updated that parametrize the new trajectory (dotted black curve).

(fixed both endpoints) can be solved in closed form in [6]. This particular construction of the prior leads to a Gauss-Markov model that generates a GP with exactly sparse tridiagonal precision matrix

$$K^{-1} = B^T Q^{-1} B \quad (20)$$

where block diagonal matrices B, Q are defined in [6]. With this inverse kernel, they can similarly define cost functional as a tradeoff between smoothness and obstacle costs, and compute functional gradient with respect to ξ . Trajectories can be parametrized by only a sparse set of support states and can be queried at any time of interest. The main advantage of GPs is that through this kernel and sparse formulation, gaussian interpolation can be performed efficiently, as shown in Fig. 2.

4.2 Factor Graph Inference

[7] Given the Markovian structure of the trajectory ξ and sparsity of inverse kernel matrix K^{-1} , the posterior distribution $p(\xi|e)$ can be further factored such that MAP inference can be equivalently viewed as performing inference on a factor graph. Similar to Eq. 7, the posterior can be factored as

$$p(\xi|e) \propto \prod_i f_i(\Xi_i) \quad (21)$$

where f_i are factors on variable subsets Ξ_i . An example factor graph that combines the definitions of the factors is shown in Fig. 3.

Specifically, we factor the prior as

$$p(\xi) \propto f_0^p(\xi_0) f_N^p(\xi_N) \prod_{i=1}^N f_i^{gp}(\xi_i, \xi_{i+1}) \quad (22)$$

Here ξ_i represents the discretized i th configuration. $f_0^p(\xi_0)$ and $f_N^p(\xi_N)$ define the prior factor on start and goal states and $f_i^{gp}(\xi_i, \xi_{i+1})$ define GP prior factors derived in [6]. Moreover, there exists two types of obstacle factors: regular factors f_i^{obs} and interpolated obstacle factors $f_{\tau_j}^{intp}$. The likelihood $l(\xi; e)$ is the product of all obstacle factors.

$$l(\xi; e) = \prod_{i=1} [f_i^{obs}(\xi_i) \prod_{j=1} f_{\tau_j}^{intp}(\xi_i, \xi_{i+1})] \quad (23)$$

The regular obstacle factor describes the obstacle costs on a single state ξ_i and the interpolated obstacle factors are obtained through fast interpolation in the Gauss-Markov Model at time τ_j satisfying $t_i \leq \tau_j \leq t_{i+1}$. Inspired by the duality between inference and optimization, we perform a minimization on the negative log of the posterior distribution solve the MAP inference in Eq. 18

$$\begin{aligned} \xi^* &= \underset{\xi}{\operatorname{argmax}} p(\xi) l(\xi; e) \\ &= \underset{\xi}{\operatorname{argmax}} (-\log(p(\xi) l(\xi; e))) \\ &= \underset{\xi}{\operatorname{argmax}} \left(\frac{1}{2} \|\xi - \mu\|_K^2 + \frac{1}{2} \|h(\xi)\|_{\Sigma_{obs}}^2 \right) \end{aligned} \quad (24)$$

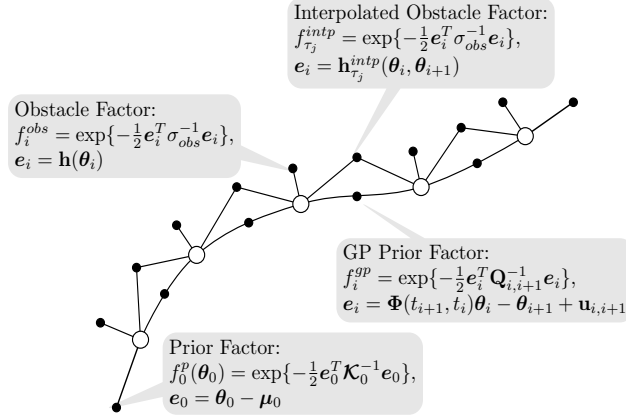


Figure 3: [7] A factor graph of an example trajectory optimization problem showing support states (white circles) and four kinds of factors (black dots), namely prior factors on start and goal states, GP prior factors that connect consecutive support states, obstacle factors on each state, and interpolated obstacle factors between consecutive support states. The θ should be interpreted as ξ since we modify the original notation. In the GP prior factor, $u_{a,b} = \int_{t_a}^{t_b} \Phi(b,s)u(s)ds$, Φ is the state transition matrix and $u(s)$, $Q_{i,i+1}$ are defined in [6]. In the obstacle factors, $h(\xi_i)$ is vector-valued obstacle cost function and σ_{obs} is a hyperparameter matrix.

143 where Σ_{obs} is the stacked obstacle hyperparameter matrix and K is the covariance matrix. Now the
 144 construction becomes a nonlinear least square problem, so iterative approaches such as Gauss-Newton
 145 can be used to solve the equation. Linearizing the cost function around the trajectory ξ ,

$$h(\xi + d\xi) \approx h(\xi) + Hd\xi \quad (25)$$

146 where H is the jacobian matrix of $h(\xi)$. Then the equation 24 becomes

$$\delta\xi^* = \underset{\delta\xi}{\operatorname{argmax}} \left(\frac{1}{2} \|\xi + \delta\xi - \mu\|_K^2 + \frac{1}{2} \|h(\xi) + H\delta\xi\|_{\Sigma_{obs}}^2 \right) \quad (26)$$

147 The optimal update $\delta\xi^*$ results from solving the linear system

$$(K^{-1} + H^T \Sigma_{obs}^{-1} H) \delta\xi^* = -K^{-1}(\xi - \mu) - H^T \Sigma_{obs}^{-1} h(\xi) \quad (27)$$

148 Once the linear system is solved, the update rule

$$\xi = \xi + \delta\xi^* \quad (28)$$

149 is applied until convergence. Again, [7] intentionally select a Gaussian process prior with a block
 150 tridiagonal precision matrix, so Eq. 8 is sparse, and $\delta\xi^*$ can be solved by exploiting the sparse
 151 Cholesky decomposition followed by forward-backward passes [3]. Combining this structure ex-
 152 ploiting iteration with the quadratic convergence rate of nonlinear least squares optimization method,
 153 GPMP2 optimize trajectory very efficiently.

154 5 Discussion and Conclusion

155 In this summary we have seen [5] and [7] that respectively formulate continuous trajectory optimiza-
 156 tion as RKHSs and GPs and reach efficient performance in practice. Their methods provide a natural
 157 solution to avoid the fine discretization requirement of previous optimization methods, which is the
 158 kernel functions that allow interpolation through fewer support states. While RKHS representation
 159 with different kernels permits the larger gradient step than less-flexible representations, GPMP2
 160 dramatically improves planning speed through modeling as Gaussian Process, inferring on factor
 161 graph, and solving least square problem efficiently.

References

- [1] Frank Dellaert, Michael Kaess, et al. Factor graphs for robot perception. *Foundations and Trends® in Robotics*, 6(1-2):1–139, 2017.
- [2] Anca D Dragan, Nathan D Ratliff, and Siddhartha S Srinivasa. Manipulation planning with goal sets using constrained trajectory optimization. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4582–4588, 2011.
- [3] Gene H Golub and Charles F Van Loan. Matrix computations , 2012. *Google Scholar*, pages 111263–112264, 2012.
- [4] Mrinal Kalakrishnan, Sachin Chitta, Evangelos Theodorou, Peter Pastor, and Stefan Schaal. Stomp: Stochastic trajectory optimization for motion planning. In *2011 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4569–4574, 2011.
- [5] Z Marinho, A Dragan, A Byravan, B Boots, G Gordon, and S Srinivasa. Functional gradient motion planning in reproducing kernel hilbert spaces. *arXiv preprint arXiv:1601.03648*, 2016.
- [6] Mustafa Mukadam, Xinyan Yan, and Byron Boots. Gaussian process motion planning. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 9–15. IEEE, 2016.
- [7] Mustafa Mukadam, Jing Dong, Xinyan Yan, Frank Dellaert, and Byron Boots. Continuous-time gaussian process motion planning via probabilistic inference. *The International Journal of Robotics Research (IJRR)*, 37(11):1319–1340, 2018.
- [8] Nathan Ratliff, Matt Zucker, J Andrew Bagnell, and Siddhartha Srinivasa. Chomp: Gradient optimization techniques for efficient motion planning. *IEEE International Conference on Robotics and Automation (ICRA)*, 2009.
- [9] John Schulman, Yan Duan, Jonathan Ho, Alex Lee, Ibrahim Awwal, Henry Bradlow, Jia Pan, Sachin Patil, Ken Goldberg, and Pieter Abbeel. Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research (IJRR)*, 33(9):1251–1270, 2014.